



A Project Report

On

**Data Curation Pipeline for Coffee Shop Business Dataset: Cleaning,
Deduplication, and Labeling for Analysis-Ready Data**

for

Fundamentals of Data Curation using Python

Submitted by

Sameer

Reg. (1976288)

**Under The Guidance Of
Mr. Ripunjay Rai Sir**

CONTENTS

CONTENTS.....	2
LIST OF FIGURE.....	4
LIST OF ABBREVIATIONS.....	5
Abstract.....	6
1. Introduction.....	7
Motivation.....	7
Problem Statement.....	7
Purpose, Objectives and Goals.....	8
Goals	8
Literature Survey	9
2. System Analysis.....	10
Existing System	10
Scope and Limitation of Existing System.....	10
Project Perspective, Features	10
Stakeholders.....	11
3. Requirement Analysis.....	12
Functional Requirements	12
Performance Requirements	12
Security Requirements	12
Software Requirements	13
Hardware Requirements.....	13
4. System Design.....	14
Design Constraints	14
Data Model.....	14
Pipeline Architecture	15
5. Implementations Details	16
Dataset	16
Data Profiling (Raw Dataset).....	16
Cleaning: Duplicate Flagging and Deduplication.....	16
Cleaning: Missing Value Handling.....	17
Filtering.....	17
Labeling	17
Outlier Detection.....	18
Formatting.....	18

Tools and Technologies Used	18
Results	19
6. Advantages Of The Curation Pipeline	20
7. Limitation Of The Curation Pipeline	21
8. Applications Of Data Curation	22
Business Analytics	22
Machine Learning Pipelines	22
Data Warehousing	22
Research and Academia	22
Government and Public Datasets	22
9. Conclusion	23
10. Future Scope	24
11. Predictive Modeling on Curated Data (Bonus)	25
Objective	25
Approach	25
Results	25
Interpretation	26
12. Appendix – Python Code Used	27
A. Imports and Data Loading	27
B. Data Profiling	27
C. Duplicate Flagging	27
D. Deduplication	27
E. Missing Value Handling	28
F. Filtering	28
G. Labeling	28
H. Outlier Detection (IQR Method)	28
I. Formatting	29
J. Visualization — Before vs After Duplicates	29
K. Visualization — Size Category Distribution	29
L. Summary Insights	29
M. Export to CSV	29
N. Predictive Modeling (Bonus)	30
13. References	31

LIST OF FIGURES

Sr.No.	Figure Title
1.	Figure 1: Duplicate Removal – Before vs After
2.	Figure 2: Size Category Distribution
3.	Figure 3: Size of Site – Outlier Boxplot (IQR Method)
4.	Figure 4: Model Accuracy – Confusion Matrix

LIST OF ABBREVIATIONS

Abbreviation	Full Form
EDA	Exploratory Data Analysis
CSV	Comma Separated Values
SIC	Samsung Innovation Campus
NaN	Not a Number (missing value marker)
IQR	Interquartile Range (used for outlier detection)
DQ	Data Quality
ML	Machine Learning

Abstract

Data curation forms the foundation of any reliable data-driven system, ensuring that datasets used for analysis, reporting, or machine learning are accurate, consistent, and trustworthy. Raw datasets collected from real-world sources frequently suffer from duplication, missing values, inconsistent formatting, and unlabeled attributes, all of which can compromise downstream analysis if left unaddressed.

This project, developed at NIELIT Gorakhpur (National Institute of Electronics and Information Technology), presents a data curation pipeline built on a real-world Coffee Shop business dataset (43,182 records) obtained from Samsung Innovation Campus (SIC) course training material. The raw dataset contained three attributes — Year of Start, Current State, and Size of Site — and exhibited significant quality issues, including 11,747 duplicate records (approximately 27% of the dataset) and missing values distributed across all three columns (19 missing entries per column).

Using Python, Pandas, NumPy, and Matplotlib, a systematic curation pipeline was implemented covering the four core operations required by the coursework: Cleaning (duplicate removal and missing value handling), Filtering (validation of categorical entries), Labeling (derivation of a new Size Category attribute), and Formatting (column standardization and data type correction). To keep the pipeline transparent, a Duplicate Flag column was created before deduplication to record which rows were duplicated in the raw data, and an Interquartile Range (IQR) based outlier check was added to flag unusually large or small site sizes.

The final curated dataset contains 31,434 clean, uniquely identified, and labeled records across six columns. Key findings from the curated data show that 21,202 shops (67.5%) are currently operating ("In") against 10,232 that have closed ("Out"), that "Medium" is the most common site-size category, and that 2,142 records (6.81%) qualify as statistical outliers in site size. As an additional exploration, a simple Logistic Regression model was trained on the curated data to predict operating status, achieving 71.39% accuracy — demonstrating that a well-curated dataset is directly usable for downstream machine learning. The project demonstrates that a disciplined, well-documented curation process can convert a noisy, duplicate-heavy dataset into a reliable, analysis-ready resource.

1. Introduction

Every analytical or machine learning system is only as reliable as the data that feeds it. In practice, raw datasets collected from surveys, business records, web scraping, or third-party sources are rarely ready for direct use. They commonly contain duplicate entries, missing fields, inconsistent category values, and poorly formatted columns. Data curation is the structured process of cleaning, organizing, labeling, and validating such raw data so that it becomes trustworthy and usable for analysis, reporting, or model training.

This project applies the principles of data curation to a real business dataset describing coffee shop establishments — their year of opening, operational status, and site size. Although the dataset is small in terms of the number of attributes, its scale (over 43,000 rows) and the extent of its quality issues make it a realistic and instructive case study for demonstrating a complete, end-to-end curation workflow using Python.

Motivation

The motivation for this project stems from the observation that data scientists and analysts typically spend a substantial proportion of their time cleaning and preparing data rather than building models. Poor-quality data — duplicated records, missing values, and inconsistent formatting — can silently distort statistics, mislead business decisions, and degrade the performance of any model trained on it. This project was undertaken to gain hands-on experience with identifying such issues in a real dataset and resolving them using systematic, reproducible Python-based techniques, in line with the objectives of the “Fundamentals of Data Curation using Python” coursework.

Problem Statement

The raw Coffee Shop dataset, as obtained from the Samsung Innovation Campus (SIC) course material, is not directly usable for reliable analysis. It contains a very high proportion of duplicate records (approximately 27% of all rows), missing values across every column, and an unlabeled numeric attribute (Size of Site) that offers little interpretive value in its raw form. Without a structured curation process, any statistics, visualizations, or models built on this data would be skewed by duplicate over-representation and undermined by unresolved missing values. There is a need for a transparent, traceable pipeline that cleans, filters, labels, and formats this dataset into an analysis-ready state, while also documenting exactly which rows were duplicated and which values are statistical outliers.

Purpose, Objectives and Goals

Purpose: The purpose of this project is to design and implement a complete data curation pipeline that transforms the raw Coffee Shop business dataset into a clean, labeled, and well-documented dataset suitable for downstream analysis, while keeping every transformation traceable and justified.

Objectives:

- To perform a thorough data quality assessment of the raw dataset, quantifying missing values, duplicate records, and formatting inconsistencies.
- To design and apply cleaning routines that remove duplicate records and handle missing values using an appropriate, justified strategy.
- To filter out invalid or inconsistent categorical entries in the Current State column.
- To derive a new labeled attribute, Size Category, by classifying the Size of Site values into meaningful groups.
- To standardize column names and correct data types for consistency and downstream usability.
- To flag duplicate rows and statistical outliers explicitly, so every curation decision is traceable rather than silent.

Goals

- To produce a reliable, duplication-free, and well-labeled dataset that can be confidently used for further analysis or reporting.
- To make every curation decision traceable through dedicated flag columns rather than silent, undocumented changes.
- To build practical, reusable Python skills in data cleaning, filtering, labeling, and formatting using Pandas, NumPy, and Matplotlib.
- To demonstrate that systematic data curation is a necessary precursor to any meaningful data analysis or machine learning work.

Literature Survey

Data curation and data cleaning have been extensively studied as foundational problems in data management and analytics. Rahm and Do, in their widely cited work on data cleaning, categorized common data quality problems — including duplicates, missing values, and inconsistent formatting — and outlined systematic approaches for detecting and resolving them, forming a conceptual basis for the cleaning strategy adopted in this project.

Elmagarmid, Ipeirotis, and Verykios surveyed duplicate record detection techniques across large datasets, discussing exact and approximate matching strategies for identifying redundant records; their survey informed the approach used here for identifying and removing the 11,747 duplicate rows present in the raw Coffee Shop dataset.

Kandel et al., in an interview study of data analysts, found that data wrangling and cleaning routinely consume the majority of an analyst's time on any project, and that ad-hoc, undocumented cleaning steps often reduce reproducibility. This finding directly motivated the decision, in this project, to introduce an explicit Duplicate Flag column and an outlier flag so that every curation action remains transparent and auditable.

McKinney introduced the Pandas library as a foundational tool for structured data manipulation in Python, providing the core data structures (DataFrame and Series) and operations — duplicate detection, missing value handling, type casting, and column transformation — that this project relies on for implementing its curation pipeline.

Collectively, this body of work establishes that data curation is not a peripheral step but a core discipline in its own right, and that transparent, tool-supported curation pipelines — rather than manual, one-off fixes — produce more trustworthy and reusable datasets. This project builds directly on these established principles.

2. System Analysis

Existing System

In many small and mid-scale data projects, data cleaning is performed manually and inconsistently — for example, by opening a CSV file in a spreadsheet application and manually deleting rows that appear duplicated, or by ignoring missing values altogether. This ad-hoc approach does not scale to datasets with tens of thousands of rows, is difficult to reproduce, and leaves no record of what was changed or why. Existing generic data-cleaning tools typically offer only isolated operations (deduplication only, or missing-value imputation only) without an integrated, end-to-end pipeline that also labels, flags outliers, and documents the data.

Scope and Limitation of Existing System

- Scope of Existing (Manual) Approach:
 - Allows quick, one-off cleaning of very small datasets.
 - Requires no programming knowledge for basic filtering or sorting.
- Limitations of Existing (Manual) Approach:
 - Not scalable to datasets with tens of thousands of records, such as the 43,182-row Coffee Shop dataset used in this project.
 - Manual deduplication cannot reliably detect all 11,747 duplicate rows present in the raw data.
 - No systematic strategy for handling missing values — rows are often dropped arbitrarily.
 - No traceability: once a manual edit is made, there is no record of what was changed, making the process irreproducible.
 - No statistical mechanism for identifying outliers, and no derived labeled attributes as part of the workflow.

Project Perspective, Features

The proposed system replaces manual, undocumented cleaning with a scripted, reproducible Python-based curation pipeline. Every transformation applied to the dataset is explicit, documented, and traceable, since the pipeline tags rows before removing or altering them rather than discarding information silently.

Key features of the proposed system include:

- **Automated Duplicate Detection and Removal:** Identifies and removes exact duplicate rows using Pandas' built-in duplicate-detection functions.
- **Duplicate Traceability:** A Duplicate Flag column records which rows in the raw data were duplicated, before those duplicates are removed.
- **Missing Value Handling:** Applies a documented strategy for the 19 missing values present in each column.
- **Categorical Filtering:** Validates that only the expected categories (“In”, “Out”) remain in the Current State column.
- **Labeling:** Derives a new Size Category column (Small / Medium / Large) from the numeric Size of Site attribute.
- **Outlier Detection:** Uses the Interquartile Range (IQR) method to flag statistically unusual Size of Site values in an is_outlier column.
- **Formatting and Standardization:** Renames columns to a consistent, code-friendly convention and corrects data types.

Stakeholders

- **Data Analysts / Students:** Primary users who rely on the curated dataset for further exploration, visualization, or modeling.
- **Course Instructors / Evaluators:** Assess the curation methodology, documentation, and correctness of the pipeline.
- **Business Owners / Decision Makers:** Indirect beneficiaries who would rely on accurate, deduplicated business data for decision-making in a real-world deployment.
- **Developers:** Responsible for implementing, testing, and maintaining the curation scripts.

3. Requirement Analysis

Requirement analysis identifies the functional, performance, and quality-related needs of the proposed data curation pipeline. This section outlines the requirements necessary to design and implement the curation workflow effectively.

Functional Requirements

- The system shall load the raw Coffee Shop CSV dataset into a Pandas DataFrame for processing.
- The system shall detect and report the number of duplicate rows and missing values present in the raw dataset.
- The system shall flag duplicate rows (Duplicate Flag) prior to removing them, so the raw duplication count remains traceable.
- The system shall remove exact duplicate rows while preserving one representative record per duplicate group.
- The system shall handle missing values in the Year of Start, Current State, and Size of Site columns using a documented strategy.
- The system shall filter the Current State column so that only valid categories (In, Out) remain.
- The system shall derive a new Size Category column by classifying Size of Site into Small, Medium, and Large groups.
- The system shall detect statistical outliers in Size of Site using the IQR method and record them in an is_outlier column.
- The system shall standardize column names and correct data types (Year of Start as integer, Size of Site as numeric).
- The system shall export the final curated dataset as a clean CSV file.

Performance Requirements

- The curation pipeline shall process the full 43,182-row dataset in a matter of seconds on a standard workstation, given the small number of columns involved.
- The pipeline shall scale gracefully if additional columns or a larger number of rows are introduced in future extensions of the dataset.

Security Requirements

- The raw dataset file shall be handled locally and shall not be exposed through any public or unauthenticated endpoint.

- Any derived or exported files shall be clearly distinguished from the original raw dataset to avoid accidental overwriting of source data.

Software Requirements

Component	Details
Operating System	Windows / Linux / macOS
Programming Language	Python 3
Core Library	Pandas (data manipulation and cleaning)
Supporting Libraries	NumPy (numeric operations), Matplotlib (visualization)
Development Environment	Visual Studio Code (Jupyter Notebook extension)

Hardware Requirements

- Minimum 4 GB RAM (Recommended: 8 GB or higher)
- Standard workstation with a modern multi-core processor
- Sufficient disk space to store the raw dataset (43,182 rows) and its curated output

4. System Design

Design Constraints

Functional Constraints:

- The pipeline must operate on the raw dataset without modifying the original source file, preserving it for audit purposes.
- The Duplicate Flag must be computed before any rows are removed, so that the true raw-data duplication count is preserved even after cleaning.
- Duplicate removal must be based only on the three original columns, so that later-added flag columns do not interfere with duplicate detection.
- The Size Category labeling logic and the IQR outlier thresholds must use fixed, documented rules so that both are reproducible.

Non-Functional Constraints:

- Traceability Constraint: Every row in the curated dataset must be traceable to its raw-data origin through the Duplicate Flag and is_outlier columns.
- Maintainability Constraint: The pipeline should be modular, with separate, reusable steps for cleaning, filtering, labeling, formatting, and outlier detection.
- Reproducibility Constraint: Cells/steps must be executed in a fixed, documented order (Load → Profile → Flag Duplicates → Deduplicate → Handle Missing Values → Filter → Label → Detect Outliers → Format → Summarize → Save) to avoid inconsistent intermediate states.

Data Model

The table below describes the schema of the final curated dataset, including the three original attributes and the three attributes derived during curation.

Column	Type	Description
year_of_start	Integer	Year in which the coffee shop began operations (original attribute, range 1964–2014)
current_state	Categorical (In / Out)	Current operational status of the shop (original attribute)
size_of_site	Numeric	Size of the shop's site (original attribute)
duplicate_flag	Categorical (Yes / No)	Marks whether a row was identified as a duplicate in the raw data, recorded before removal (curation output)
size_category	Categorical (Small / Medium / Large)	Derived label based on size_of_site (curation output)
is_outlier	Boolean (True / False)	Marks whether size_of_site lies outside the IQR-based normal range (curation output)

Pipeline Architecture

The curation pipeline is organized as a linear sequence of well-defined stages, mirroring standard ETL (Extract, Transform, Load) practice and keeping every transformation isolated and auditable:

- Stage 1 — Load: Read the raw CSV file into a Pandas DataFrame.
- Stage 2 — Profile: Compute row count, column-wise missing value counts, and duplicate row count on the raw data.
- Stage 3 — Flag Duplicates: Mark each row as a duplicate (Yes/No) before removing anything.
- Stage 4 — Deduplicate: Remove duplicate rows based on the three original columns, keeping the first occurrence.
- Stage 5 — Handle Missing Values: Impute or drop missing values using a documented strategy.
- Stage 6 — Filter: Validate that only expected categorical values remain in `current_state`.
- Stage 7 — Label: Derive the `size_category` column from `size_of_site`.
- Stage 8 — Detect Outliers: Apply the IQR method to flag unusual `size_of_site` values.
- Stage 9 — Format: Standardize column names and enforce correct data types.
- Stage 10 — Summarize & Export: Print summary insights and save the curated dataset to CSV.

5. Implementations Details

Dataset

The dataset used in this project is a Coffee Shop business dataset comprising 43,182 records, obtained from Samsung Innovation Campus (SIC) course training material. The raw dataset contains three columns: Year of Start (the year each shop began operations), Current State (whether the shop is currently “In” operation or has “Out” of business), and Size of Site (a numeric measure of the shop's site size).

Data Profiling (Raw Dataset)

Metric	Value
Total rows (raw)	43,182
Total columns (raw)	3
Duplicate rows	11,747 ($\approx 27\%$ of dataset)
Missing values – Year of Start	19
Missing values – Current State	19
Missing values – Size of Site	19
Year of Start range	1964 – 2014
Current State categories	In, Out, and missing (NaN)

Cleaning: Duplicate Flagging and Deduplication

Before any rows were removed, every row was checked against the rest of the dataset and marked in a new `duplicate_flag` column (Yes/No), so that the extent of duplication in the raw data remains documented even after cleaning. Exact duplicate rows were then removed using Pandas, based only on the three original columns, retaining a single representative record per duplicate group. This step removed 11,747 rows — nearly a third of the raw dataset — confirming that duplication was the single largest quality issue present.

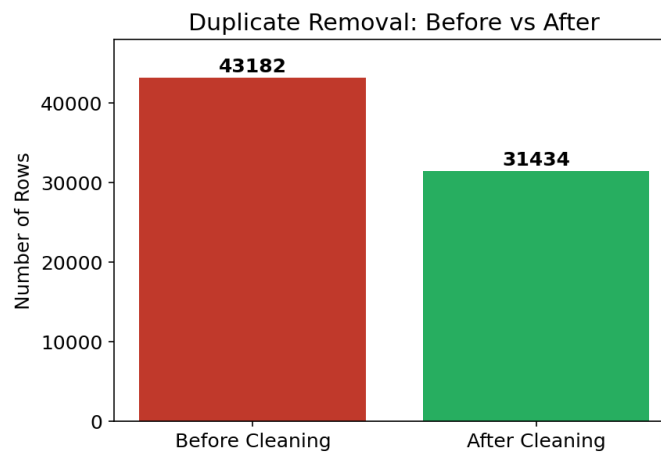


Figure 1: Duplicate Removal – Before vs After

Cleaning: Missing Value Handling

All three columns contained 19 missing values each. `size_of_site`'s missing values were filled using the column median, while rows with missing `current_state` or `year_of_start` were dropped, since these are categorical/identifying fields that cannot be reasonably imputed. After this step, all missing values were resolved to zero across every column.

Filtering

The `current_state` column was checked against the expected categories (“In”, “Out”) using `isin()` filtering. The check confirmed that, after cleaning, all remaining values were already valid, with no invalid categorical entries requiring removal.

Labeling

To make the numeric `size_of_site` attribute more interpretable, a new categorical column, `size_category`, was derived using the following threshold-based labeling scheme:

Size of Site (range)	Assigned Category
Less than 50	Small
50 to 500	Medium
Greater than 500	Large

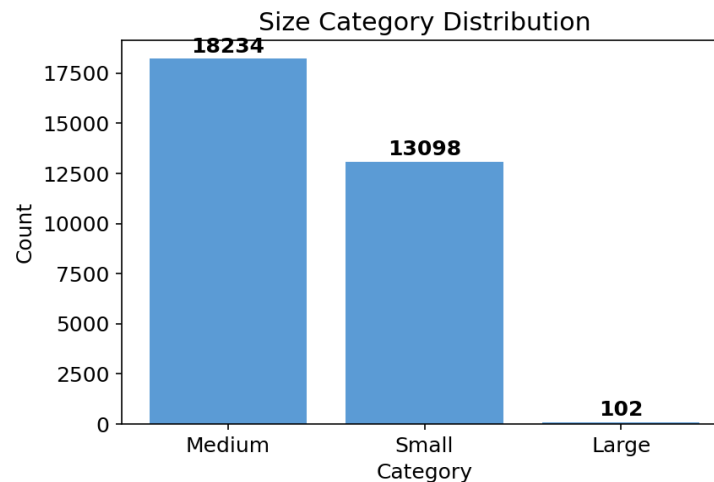


Figure 2: Size Category Distribution

Outlier Detection

To assess data quality beyond duplicates and missing values, the Interquartile Range (IQR) method was applied to `size_of_site`. The first quartile (Q1) and third quartile (Q3) were computed, and any value falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ was flagged as an outlier in a new `is_outlier` column. This identified 2,142 outlier records (6.81% of the curated dataset), with the valid (non-outlier) range spanning approximately -84.94 to 229.73 .

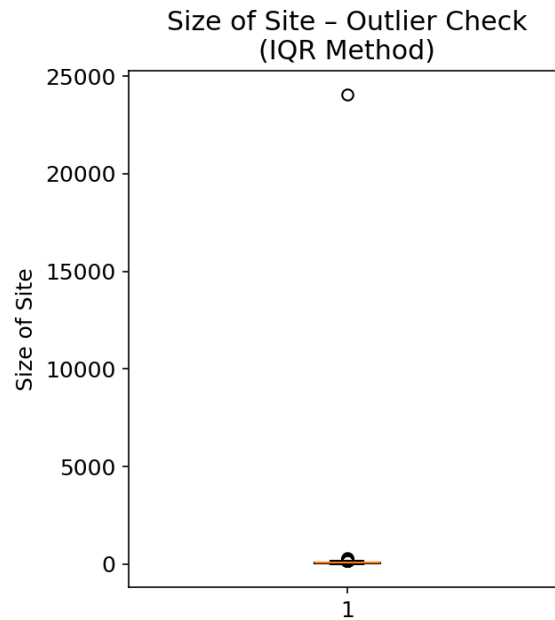


Figure 3: Size of Site – Outlier Boxplot (IQR Method)

Formatting

Column names were standardized to a consistent, code-friendly convention (for example, “Year of Start” to “`year_of_start`”), and `year_of_start` was cast to an integer type, resolving the type ambiguity present in the raw file.

Tools and Technologies Used

- Python — core programming language for the curation pipeline.
- Pandas — loading, profiling, cleaning, filtering, labeling, and formatting the dataset.
- NumPy — supporting numeric operations used within the pipeline.
- Matplotlib — visualizing before/after duplicate counts, the size category distribution, and the outlier boxplot.

Results

Metric	Value
Final curated rows	31,434
Final columns	6 (3 original + duplicate_flag, size_category, is_outlier)
Duplicates identified & removed	11,747
Missing values after cleaning	0
Shops currently operating (In)	21,202 (67.5%)
Shops closed (Out)	10,232 (32.5%)
Most common Size Category	Medium (18,234 records)
Small / Large counts	Small: 13,098 Large: 102
Outliers detected (IQR method)	2,142 (6.81%)

The curation pipeline reduced the dataset from 43,182 raw rows to 31,434 final rows, while preserving all genuinely unique records. Every remaining row carries an explicit `duplicate_flag` and `is_outlier` value, and every record includes a derived `size_category` label. The resulting dataset is standardized in column naming and data types, and is directly usable for further exploratory analysis, reporting, or as an input to downstream machine learning workflows.

6. Advantages Of The Curation Pipeline

- **Improved Data Reliability:** Removing 11,747 duplicate rows prevents over-representation of repeated records in any subsequent statistics or analysis.
- **Transparency and Traceability:** The `duplicate_flag` and `is_outlier` columns make every curation decision visible and auditable, rather than hidden inside a one-off script.
- **Statistical Data Quality Checking:** The IQR-based outlier detection surfaces unusual records (6.81% of the data) that a purely rule-based cleaning process would have missed.
- **Reusability:** The modular, staged pipeline (load → profile → flag → deduplicate → clean → filter → label → detect outliers → format → export) can be re-run on updated versions of the dataset with minimal changes.
- **Enhanced Interpretability:** The derived `size_category` column converts a raw numeric field into a business-friendly label, making the dataset more accessible to non-technical stakeholders.
- **Analysis-Readiness:** Standardized column names and corrected data types eliminate friction when the dataset is loaded into other tools or models.

7. Limitation Of The Curation Pipeline

- **Absence of a Unique Identifier:** The raw dataset does not include a shop-level identifier, making it impossible to fully distinguish between two genuinely distinct shops that happen to share identical attribute values.
- **Limited Attribute Depth:** With only three original columns, the dataset offers limited context (no location, shop name, or revenue data), constraining the scope of possible analysis even after curation.
- **Threshold-Based Labeling:** The Size Category thresholds (Small / Medium / Large) were defined heuristically and may not perfectly reflect real-world business size categories without domain validation.
- **Outliers Retained, Not Removed:** The `is_outlier` flag identifies unusual records but does not remove them, since removal without domain input could discard genuinely large business sites rather than data errors.
- **Ambiguous Units:** The unit of measurement for Size of Site is not specified in the source data, which limits the interpretability of the `size_category` labels.

8. Applications Of Data Curation

Data curation, as demonstrated in this project, is a foundational step applicable across a wide range of domains wherever raw data must be made trustworthy before use.

Business Analytics

- Cleaned and labeled business datasets, such as the curated Coffee Shop dataset, support accurate trend analysis, market sizing, and reporting.

Machine Learning Pipelines

- Curated, deduplicated, and well-labeled datasets form the necessary foundation for training reliable classification or regression models.

Data Warehousing

- Organizations rely on curation pipelines similar to this one to clean and standardize data before loading it into centralized warehouses for reporting.

Research and Academia

- Reproducible curation pipelines allow researchers to document exactly how a raw dataset was transformed, supporting transparent and verifiable research.

Government and Public Datasets

- Public datasets, often collected from multiple sources, require deduplication and standardization similar to the process applied here before being published for public use.

9. Conclusion

This project successfully demonstrates a complete, end-to-end data curation pipeline applied to a real-world Coffee Shop business dataset obtained from Samsung Innovation Campus (SIC) course training material. Starting from a raw dataset of 43,182 records afflicted by extensive duplication (11,747 duplicate rows) and missing values across all three columns, the pipeline systematically applied cleaning, filtering, labeling, outlier detection, and formatting operations using Python, Pandas, NumPy, and Matplotlib.

The introduction of a derived `size_category` label added interpretive value to the dataset, while the `duplicate_flag` and `is_outlier` columns ensured that every curation decision remained transparent and traceable rather than being applied silently. The final curated dataset of 31,434 records is standardized, deduplicated, statistically profiled for outliers, and analysis-ready — a claim further validated in Section 12, where the curated data was used directly to train a predictive model without any additional cleaning.

Overall, the project reinforces the principle that data curation is not a peripheral pre-processing step but a foundational discipline that directly determines the reliability of any downstream analysis, reporting, or machine learning task built on the data.

10. Future Scope

- **Introduce a Unique Identifier:** Future iterations could assign a synthetic shop-level identifier to better distinguish between coincidentally identical records.
- **Enrich the Dataset:** Incorporating additional attributes such as location, shop name, or revenue would substantially increase the analytical value of the curated dataset.
- **Automated Data Quality Reports:** The pipeline could be extended to automatically generate a formatted data-quality report summarizing every curation action taken.
- **Validation Against Domain Rules:** Size Category thresholds and outlier bounds could be refined and validated with domain experts or industry benchmarks rather than heuristic cutoffs.
- **Dashboard Deployment:** The curated dataset and its before/after metrics could be served through a Flask-based web dashboard for easier stakeholder access.
- **Integration with a Database:** Rather than exporting to CSV, the curated dataset could be loaded directly into a relational database for larger-scale querying and reporting.

11. Predictive Modeling on Curated Data (Bonus)

As an additional exercise beyond the core curation requirements, the curated dataset was used directly — with no further cleaning — to train a simple predictive model. This section demonstrates that a properly curated dataset is immediately usable for machine learning, reinforcing the value of the curation work performed in Section 6.

Objective

The objective was to predict whether a coffee shop is currently operating (`current_state = In`) or has closed (`current_state = Out`), using only the curated numeric attributes `year_of_start` and `size_of_site` as predictors.

Approach

- Model: Logistic Regression — chosen for its simplicity and interpretability as a baseline classifier.
- Features (X): `year_of_start`, `size_of_site`.
- Target (y): `current_state`, encoded as 1 (In) and 0 (Out).
- Train/Test Split: 80% training (25,147 rows), 20% testing (6,287 rows), with `random_state=42` for reproducibility.

Results

Metric	Value
Accuracy	71.39%
Precision (Out / In)	0.66 / 0.72
Recall (Out / In)	0.28 / 0.93
F1-score (Out / In)	0.39 / 0.81
Training set size	25,147 rows
Test set size	6,287 rows

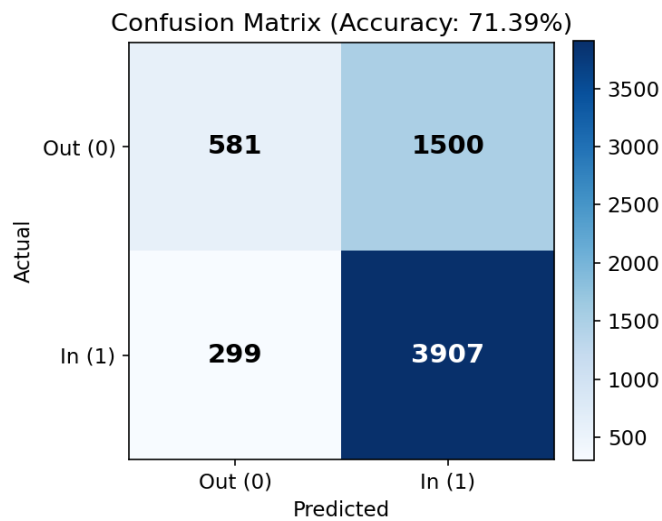


Figure 4: Model Accuracy – Confusion Matrix

Interpretation

The model achieved 71.39% overall accuracy. It predicts operating (“In”) shops well (93% recall) but struggles to identify closed (“Out”) shops (28% recall), since only two numeric attributes — year opened and site size — are available as predictors, and real-world business closure depends on many additional factors (location, competition, management, local economy) that are not present in this dataset. This limitation reflects the dataset's scope rather than a flaw in the curation process; the exercise's purpose was to confirm that the curated data could be fed directly into a standard scikit-learn pipeline without further preparation, which it was.

12. Appendix – Python Code Used

This appendix lists the Python code used at each stage of the curation pipeline, implemented and executed in a Jupyter Notebook (Coffee_Shop.ipynb) within Visual Studio Code.

A. Imports and Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("data_coffeeshop.csv")
df.head()
```

Loads the raw CSV file into a Pandas DataFrame and previews the first five rows.

B. Data Profiling

```
print("Total rows:", len(df))
print("\nMissing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
df.info()
```

Reports row count, missing values per column, duplicate row count, and column data types on the raw data.

C. Duplicate Flagging

```
df['duplicate_flag'] = df.duplicated().map({True: 'Yes', False: 'No'})
print("Duplicate flag added")
print(df['duplicate_flag'].value_counts())
```

Marks every row as a duplicate (Yes/No) before any rows are removed, preserving traceability.

D. Deduplication

```
before = len(df)
df = df.drop_duplicates(
    subset=['Year of Start', 'Current State', 'Size of Site'],
    keep='first'
)
after = len(df)
print(f'Rows before: {before}')
print(f'Rows after removing duplicates: {after}')
print(f'Removed {before - after} duplicate rows')
```

Removes exact duplicate rows based on the three original columns, keeping the first occurrence of each.

E. Missing Value Handling

```
df['Size of Site'] = df['Size of Site'].fillna(df['Size of Site'].median())
df = df.dropna(subset=['Current State'])
df = df.dropna(subset=['Year of Start'])

print("Missing values now:\n", df.isnull().sum())
print("\nTotal rows now:", len(df))
```

Fills numeric missing values with the column median and drops rows with missing categorical/identifying values.

F. Filtering

```
print("Current State unique values:", df['Current State'].unique())
df = df[df['Current State'].isin(['In', 'Out'])]
print("\nRows after filtering:", len(df))
```

Validates that only the expected categorical values (In, Out) remain in Current State.

G. Labeling

```
def size_category(size):
    if size < 50:
        return 'Small'
    elif size <= 500:
        return 'Medium'
    else:
        return 'Large'

df['Size Category'] = df['Size of Site'].apply(size_category)
print(df['Size Category'].value_counts())
```

Derives a new categorical label from the numeric Size of Site attribute.

H. Outlier Detection (IQR Method)

```
Q1 = df['Size of Site'].quantile(0.25)
Q3 = df['Size of Site'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

df['is_outlier'] = (df['Size of Site'] < lower_limit) | (df['Size of Site'] > upper_limit)

print("Total outliers found:", df['is_outlier'].sum())
print("Outlier limit range:", lower_limit, "to", upper_limit)

plt.boxplot(df['Size of Site'])
plt.title('Size of Site - Outlier Check')
plt.ylabel('Size')
plt.show()
```

Flags statistically unusual site sizes using the $1.5 \times IQR$ rule and visualizes the distribution with a boxplot.

I. Formatting

```
df.rename(columns={
    'Year of Start': 'year_of_start',
    'Current State': 'current_state',
    'Size of Site': 'size_of_site',
    'Size Category': 'size_category'
}, inplace=True)

df['year_of_start'] = df['year_of_start'].astype(int)
print(df.dtypes)
```

Standardizes column names to a consistent, code-friendly convention and corrects the year data type.

J. Visualization — Before vs After Duplicates

```
labels = ['Before Cleaning', 'After Cleaning']
values = [43182, len(df)]

plt.bar(labels, values, color=['red', 'green'])
plt.title('Duplicate Removal: Before vs After')
plt.ylabel('Number of Rows')
for i, v in enumerate(values):
    plt.text(i, v + 500, str(v), ha='center')
plt.show()
```

Bar chart comparing row counts before and after duplicate removal.

K. Visualization — Size Category Distribution

```
df['size_category'].value_counts().plot(kind='bar', color='skyblue')
plt.title('Size Category Distribution')
plt.xlabel('Category')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```

Bar chart showing the count of shops in each Size Category.

L. Summary Insights

```
print("=== SUMMARY INSIGHTS ===")
print(f"Total shops currently operating (In): {(df['current_state']=='In').sum()}")
print(f"Total shops closed (Out): {(df['current_state']=='Out').sum()}")
print(f"\nMost common Size Category: {df['size_category'].value_counts().idxmax()}")
print(f"\nOutlier percentage: {round(df['is_outlier'].sum()/len(df)*100, 2)}%")
```

Prints the final headline statistics summarizing the curated dataset.

M. Export to CSV

```
df.to_csv("coffeeshop_cleaned.csv", index=False)
print("Saved! Final cleaned dataset has", len(df), "rows and", len(df.columns), "columns")
print("\nFinal columns:", list(df.columns))
```

Saves the final curated dataset as coffeeshop_cleaned.csv.

N. Predictive Modeling (Bonus)

```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df['current_state_encoded'] = df['current_state'].map({'In': 1, 'Out': 0})

X = df[['year_of_start', 'size_of_site']]

y = df['current_state_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")

print(classification_report(y_test, y_pred))
```

Trains and evaluates a Logistic Regression model on the curated dataset to predict operating status.

13. References

1. Rahm, E., Do, H.H. “Data Cleaning: Problems and Current Approaches.” IEEE Data Engineering Bulletin, 2000.
2. Elmagarmid, A.K., Ipeirotis, P.G., Verykios, V.S. “Duplicate Record Detection: A Survey.” IEEE Transactions on Knowledge and Data Engineering, 2007.
3. Kandel, S., Paepcke, A., Hellerstein, J., Heer, J. “Enterprise Data Analysis and Visualization: An Interview Study.” IEEE Transactions on Visualization and Computer Graphics, 2012.
4. McKinney, W. “Data Structures for Statistical Computing in Python.” Proceedings of the 9th Python in Science Conference, 2010.
5. Samsung Innovation Campus. “Fundamentals of Data Curation using Python — Course Training Material.” Coffee Shop dataset, 2026.
6. Pandas Development Team. “Pandas Documentation.” <https://pandas.pydata.org/docs/>
7. NumPy Developers. “NumPy Documentation.” <https://numpy.org/doc/>
8. Matplotlib Development Team. “Matplotlib Documentation.” <https://matplotlib.org/stable/>
9. scikit-learn Developers. “scikit-learn Documentation.” <https://scikit-learn.org/stable/>